HelixCode Step-by-Step Setup & Usage Guide

HelixCode - Complete Setup, Build, and Usage Guide

Version: 1.0.0 **Date**: 2026-04-30 **Prerequisites**: Go 1.24.0+, Docker (optional), PostgreSQL 15+ (optional), Redis 7+ (optional)

Table of Contents

---

1. Environment Setup

1.1 Install Go

```
# Check current Go version
go version

# Should show: go version go1.24.0 or later
# If not, install Go:

# Linux (Ubuntu/Debian)
wget https://go.dev/dl/go1.24.0.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.24.0.linux-amd64.tar.gz
export PATH=$PATH:/usr/local/go/bin

# macOS
brew install go@1.24

# Verify
go version
# Expected: go version go1.24.0 linux/amd64
```

1.2 Install Docker (Optional but Recommended)

```
# Docker installation varies by platform
# See: https://docs.docker.com/get-docker/
```

```
# Verify installation
docker --version
docker-compose --version
```

1.3 Install PostgreSQL (Optional)

```
# Ubuntu/Debian
sudo apt-get update
sudo apt-get install postgresql postgresql-contrib

# macOS
brew install postgresql
brew services start postgresql

# Verify
psql --version
```

1.4 Install Redis (Optional)

```
# Ubuntu/Debian
sudo apt-get install redis-server
sudo systemctl start redis

# macOS
brew install redis
brew services start redis

# Verify
redis-cli ping
# Expected: PONG
```

---

2. Repository Setup

2.1 Clone HelixCode

```
# IMPORTANT: Use SSH (per Constitution CONST-003)
git clone git@github.com:HelixDevelopment/HelixCode.git
cd HelixCode

# Initialize submodules
git submodule update --init --recursive

# Verify submodules
ls Example_Projects/
# Should show: Aider, Cline, Codex, OpenHands, etc.
```

2.2 Verify Project Structure

```
# Should show key directories
```

```
ls -la
# Expected: AGENTS.md, Dockerfile, HelixCode/, docker-compose.helix.yml, etc.

# Navigate to main code directory
cd HelixCode
ls -la
# Expected: cmd/, internal/, applications/, tests/, config/, go.mod
```

CRITICAL: All build commands MUST be run from the `HelixCode/` subdirectory, NOT repository root.

2.3 Configure Environment

```
cd HelixCode
cp .env.example .env

# Edit .env with your settings
# Required for production:
# HELIX_AUTH_JWT_SECRET=your-super-secure-jwt-secret-min-32-chars
# HELIX_DATABASE_PASSWORD=your-secure-database-password
# HELIX_REDIS_PASSWORD=your-secure-redis-password

# LLM Provider Keys (at least one):
# OPENAI_API_KEY=sk-your-key
# ANTHROPIC_API_KEY=sk-ant-your-key
# Or use free providers (GitHub Copilot with subscription)
```

2.4 Fix go.mod (CRITICAL - Current Bluff)

The root `go.mod` only has 3 dependencies. You MUST expand it:

```
# View current (broken) go.mod
cat go.mod
# Shows only: uuid, errors, yaml

# Replace with proper go.mod (see HELIXCODE_ZERO_BLUFF_PLAN.md P0-001)
# Then download dependencies
go mod tidy

# Verify all imports resolve
go list -m all
```

Anti-Bluff Verification: If `go mod tidy` fails, dependencies are missing. This confirms the bluff.

---

3. Building HelixCode

3.1 Generate Logo Assets (Required First Time)

```
cd HelixCode
make logo-assets
```

```
# Verify assets created
ls assets/
# Expected: logo.png, logo.svg, etc.
```

3.2 Build the Application

```
# Standard build
make build

# Verify binaries created
ls bin/
# Expected: server, cli, terminal-ui, etc.

# Build with verbose output
make build V=1
```

3.3 Verify Build Integrity

```
# Check binary version
./bin/server --version
# Expected: HelixCode v1.0.0 (or current version)

# Check CLI help
./bin/cli --help
# Expected: List of all flags and commands
```

Anti-Bluff Verification: If build fails, the "fully complete" claim is a bluff.

---

4. Database Setup

4.1 Create Database (Optional)

```
# Start PostgreSQL service
sudo systemctl start postgresql

# Create database and user
sudo -u postgres createdb helixcode_prod
sudo -u postgres createuser helix
sudo -u postgres psql -c "ALTER USER helix WITH PASSWORD 'your_password';"

# Verify connection
psql -U helix -d helixcode_prod -c "SELECT 1;"
# Expected: 1 row with value 1
```

4.2 Run Migrations (If Available)

```
# Check if migrations exist
ls internal/database/migrations/
```

```
# If they exist, run them
make migrate
# Or manually:
# psql -U helix -d helixcode_prod -f internal/database/migrations/001_initial.sql
```

Anti-Bluff Verification: If migrations don't exist, the "11 tables" claim is unverified.

4.3 Configure Database (Optional for Testing)

Edit `config/config.yaml`:

```
database:
  host: ""  # Empty = disabled (for testing without DB)
  # OR
  host: "localhost"
  port: 5432
  user: "helix"
  password: ""  # From HELIX_DATABASE_PASSWORD env var
  dbname: "helixcode_prod"
  sslmode: "disable"
```

Note: Database can be disabled by setting `host: ""` for testing.

---

5. Running the Server

5.1 Start the Server

```
cd HelixCode

# Load environment
export HELIX_AUTH_JWT_SECRET="your-super-secure-secret-min-32-characters"
export HELIX_DATABASE_PASSWORD="your-db-password"  # Optional
export HELIX_REDIS_PASSWORD="your-redis-password"  # Optional

# Start server
./bin/server

# Or with config file
./bin/server --config config/config.yaml
```

5.2 Verify Server Health

```
# Basic health check
curl http://localhost:8080/health

# Expected (when fully implemented):
# {
#   "status": "healthy",
#   "database": "connected",
#   "redis": "connected",
```

```
#   "providers": ["ollama", "openai"]
# }

# Check metrics
curl http://localhost:8080/metrics
```

Anti-Bluff Verification: If `/health` returns 404 or empty JSON, the endpoint is not implemented.

---

6. Using the CLI

6.1 Basic CLI Usage

```
cd HelixCode

# Interactive mode
./bin/cli

# List available commands
helix> help
```

6.2 LLM Generation (CRITICAL - Currently Simulated)

```
# Current behavior (BLUFF):
./bin/cli --prompt "What is the capital of France?" --model llama-3-8b
# Expected current output: "This is a simulated response..."
# THIS IS A BLUFF - the model is not actually called

# After zero-bluff fix, expected behavior:
./bin/cli --prompt "What is the capital of France?" --model llama-3-8b
# Expected: "Paris is the capital of France."
# Verify: Response should not contain "simulated"
```

Anti-Bluff Verification:

```
# Check if response is simulated
./bin/cli --prompt "test" 2>&1 | grep -i "simulated"
# If output contains "simulated", BLUFF CONFIRMED
```

6.3 List Models (Currently Hardcoded)

```
./bin/cli --list-models

# Current output (BLUFF):
# ID: llama-3-8b
#   Name: Llama 3 8B
#   Provider: llama.cpp
#   Status: available
# (Only 3 hardcoded models)

# After fix, expected behavior:
```

```
# Should query actual providers and show real available models
```

6.4 Worker Management

```
# List workers
./bin/cli --list-workers

# Add a worker
./bin/cli --worker worker-host.example.com --user helix --key ~/.ssh/id_rsa

# Verify worker added
./bin/cli --list-workers
```

6.5 Health Check

```
./bin/cli --health

# Expected output:
# === System Health Check ===
# Worker Pool: X healthy workers
# Notification System: X enabled channels
# System is operational
```

6.6 Send Notification

```
./bin/cli --notify "Deployment complete" --notify-type success --notify-priority medium
```

---

7. Docker Deployment

7.1 Build Docker Image

```
cd HelixCode

# Build image
docker build -t helixcode:latest .

# Verify build
docker images | grep helixcode
```

7.2 Run with Docker Compose

```
cd HelixCode

# Create environment file
cat > .env << EOF
HELIX_AUTH_JWT_SECRET=your-super-secure-jwt-secret-min-32-characters
HELIX_DATABASE_PASSWORD=your-db-password
HELIX_REDIS_PASSWORD=your-redis-password
GRAFANA_ADMIN_PASSWORD=your-grafana-password
```

```
EOF

# Start all services
docker-compose up -d

# Check status
docker-compose ps

# View logs
docker-compose logs -f helixcode-server
```

7.3 Verify Docker Deployment

```
# Check all containers are up
docker-compose ps
# Expected: helixcode-server, postgres, redis, nginx all showing "Up"

# Check health
curl http://localhost/health

# Check database
docker-compose exec postgres psql -U helix -d helixcode_prod -c "SELECT 1;"

# Check redis
docker-compose exec redis redis-cli ping
# Expected: PONG
```

Anti-Bluff Verification: If any container shows `unhealthy` or `restarting`, deployment is not complete.

7.4 Stop Docker Deployment

```
docker-compose down

# With volumes removal (WARNING: deletes data)
docker-compose down -v
```

---

8. Testing Everything

8.1 Run Unit Tests

```
cd HelixCode

# Quick unit tests (mocks allowed)
go test -short ./...

# With coverage
go test -short -cover ./...

# Specific package
```

```
go test -v -run TestAuthService ./internal/auth
```

8.2 Run Integration Tests

```
# Start test infrastructure
make test-infra-up

# Run integration tests
make integration-test

# Stop test infrastructure
make test-infra-down
```

8.3 Run Challenges (Anti-Bluff Verification)

```
cd HelixCode/tests/e2e/challenges

# Run all challenges
./run_all_challenges.sh

# Run specific challenge
./run_all_challenges.sh --filter llm_generation

# Run with verbose output
./run_all_challenges.sh --verbose
```

Expected Output:

```
Challenge: llm_generation_001
  [PASS] Response is not empty
  [PASS] Response is not simulated
  [PASS] Response contains expected answer
  [PASS] Response has reasonable length
Challenge PASSED
```

Anti-Bluff Verification: If challenges pass but features don't work, the challenges are bluffs. Tighten them.

8.4 Run Security Tests

```
make security-test
```

8.5 Run Performance Benchmarks

```
make benchmark
```

---

9. Troubleshooting

9.1 Build Failures

Problem: `make build` fails with missing dependencies

```
# Solution 1: Fix go.mod
cat go.mod
# If only 3 dependencies, apply fix from HELIXCODE_ZERO_BLUFF_PLAN.md
go mod tidy

# Solution 2: Missing logo assets
make logo-assets
make build

# Solution 3: Go version mismatch
go version
# Must be 1.24.0+
```

9.2 Database Connection Errors

Problem: Server fails to connect to PostgreSQL

```
# Check PostgreSQL is running
sudo systemctl status postgresql

# Check connection manually
psql -U helix -d helixcode_prod -c "SELECT 1;"

# If fails, check:
# 1. HELIX_DATABASE_PASSWORD env var is set
# 2. PostgreSQL is listening on correct port
# 3. User and database exist
# 4. pg_hba.conf allows local connections

# Disable database for testing
# Edit config/config.yaml: database.host = ""
```

9.3 LLM Generation Returns Simulated Response

Problem: `./bin/cli --prompt "test"` returns "This is a simulated response"

Root Cause: BLUFF-001 - LLM generation is not actually implemented

Solution: Implement real provider integration (see HELIXCODE_ZERO_BLUFF_PLAN.md Phase 1)

Immediate workaround: None. This feature is advertised but not implemented.

9.4 Worker SSH Connection Fails

Problem: Cannot add worker via SSH

```
# Verify SSH key exists
ls ~/.ssh/id_rsa
```

```
# Verify SSH connection works manually
ssh -i ~/.ssh/id_rsa helix@worker-host

# Verify worker has Go installed
ssh -i ~/.ssh/id_rsa helix@worker-host "go version"
```

9.5 Docker Container Exits Immediately

Problem: Container starts then exits

```
# Check logs
docker-compose logs helixcode-server

# Common causes:
# 1. Missing docker-entrypoint.sh (BLUFF-008)
#    Fix: Create the script (see HELIXCODE_ZERO_BLUFF_PLAN.md P0-002)
# 2. Missing environment variables
#    Fix: Check .env file exists and is populated
# 3. Database not ready
#    Fix: Add wait-for-it logic to entrypoint
```

---

10. Next Steps

For Users

1. **Verify zero-bluff status** before using in production:

   ```
   ./tests/e2e/challenges/run_all_challenges.sh
   ```

   All challenges MUST pass.

2. **Configure at least one LLM provider**:
   - Local: Install Ollama (`ollama run llama3.2`)
   - Cloud: Set OPENAI_API_KEY or ANTHROPIC_API_KEY
   - Free: Use GitHub Copilot (with subscription)

3. **Set up workers** (for distributed computing):

   ```
   ./bin/cli --worker worker1.local --user helix --key ~/.ssh/id_rsa
   ```

For Developers

1. **Read the Constitution**: `CONSTITUTION.md`
2. **Read CLAUDE.md**: `CLAUDE.md`
3. **Review gap analysis**: `HELIXCODE_GAP_ANALYSIS.md`
4. **Follow the plan**: `HELIXCODE_ZERO_BLUFF_PLAN.md`
5. **Write anti-bluff tests**: Every feature needs real tests
6. **Write challenges**: Every feature needs end-to-end validation

For Contributors

1. **No CI/CD**: All builds and tests run manually
2. **SSH only**: No HTTPS for Git operations
3. **Evidence required**: Every PR needs pasted terminal output
4. **No mocks in production**: Only unit tests may use mocks
5. **Reproduce before fix**: Write Challenge first, then fix

Quick Reference Commands

```
# Setup
git clone git@github.com:HelixDevelopment/HelixCode.git
cd HelixCode/HelixCode
cp .env.example .env
make logo-assets

# Build
make build

# Test
go test -short ./...
./tests/e2e/challenges/run_all_challenges.sh

# Run
export HELIX_AUTH_JWT_SECRET="your-secret"
./bin/server

# CLI
./bin/cli --prompt "Hello world" --model llama-3-8b

# Docker
docker-compose up -d
docker-compose ps

# Verify no bluffs
grep -r "simulated\|for now\|TODO implement\|placeholder" internal/ cmd/
# Should return NOTHING
```

This guide is verified against the actual HelixCode repository. If any step fails, the feature is a bluff and needs implementation per the zero-bluff plan.